



ASSIGNMENT 1

Subject: Operating System

Section: BSSE5-2

Submitted By:

Name: ABDUL REHMAN

SAP ID: 65236

Submitted to:

Sir Ishtiaq

Faculty of Computing,
Riphah International University, Islamabad.

Operating Systems – Assignment 01

Question 1 – Role of Operating System [3 Marks]

a. Why is the operating system a resource allocator in this situation?

In the lab, browsers, compilers, database applications and media players all run at the same time and compete for the CPU, main memory, files and I/O devices. The operating system manages all of these resources. When several programs make conflicting requests, it decides between them so that the resources are used efficiently and fairly. For example, it schedules CPU time among the running programs, allocates and deallocates memory space as needed, and uses request/release code for devices such as printers. It also keeps accounting information about which user consumes how much of each resource. Because it allocates every resource among many users and jobs, the OS acts as a resource allocator.

b. Two applications request the same I/O device at about the same time.

The operating system must make sure the device is used in a controlled and orderly way. It cannot let both applications drive the device at once, since the data would be mixed up or corrupted. The OS checks the device-status table, which records the type, address and state of each device. It grants the device to one request and places the other in a queue for that device. When the device finishes, its controller signals the CPU with an interrupt, and the OS starts the next waiting request. Applications never control the device directly; the OS does so through the device driver, which hides the hardware details. In this way it provides fair and conflict-free access to the device.

c. A program tries to access an OS resource improperly.

As a control program, the OS controls the execution of programs to prevent errors and improper use of the computer. It uses dual-mode operation: user programs run in user mode, and privileged instructions, such as direct access to hardware, can run only in kernel mode. If the student's program attempts an illegal operation or touches memory or resources it does not own, the hardware raises a trap (a software-generated interrupt). Control then passes to the operating system, which can deny the request and terminate the offending program with an error. Protection mechanisms such as user IDs and access control also ensure that one process cannot interfere with the OS or with other users' processes. As a result, the improper access is stopped and the rest of the system keeps running correctly.

Question 2 – Interrupts and I/O [4 Marks]

a. How can the CPU and an I/O device work concurrently?

Each I/O device has its own device controller, which is in charge of that particular type of device and can operate independently of the CPU. The CPU only starts the transfer by giving the controller the request. The controller then moves data between the disk and its local buffer by itself. While the disk transfers data, the CPU is free to execute another program. The CPU and the device controllers are connected through a common bus to shared memory, so they run concurrently and compete only for memory cycles.

b. Role of the device controller's local buffer.

The local buffer is a small storage area inside the device controller. I/O takes place between the device and this buffer. Data from the disk is first collected in the buffer, and the CPU then moves it between the buffer and main memory. The buffer thus compensates for the speed difference between the slow device and the fast CPU and memory, and it lets the device keep transferring without the CPU having to wait on every byte.

c. Sequence of events when the controller finishes and generates an interrupt.

1. The device controller finishes the transfer and causes an interrupt to inform the CPU.
2. The CPU stops what it is doing, and the hardware saves the address of the interrupted instruction. Further incoming interrupts are disabled while this one is processed, so that no interrupt is lost.
3. The OS saves the state of the CPU (registers and program counter).
4. The OS determines the type of interrupt, either by polling or through a vectored interrupt system. The interrupt vector holds the addresses of all the service routines.
5. Control transfers to the correct interrupt service routine (ISR), a separate segment of code that handles that device. For example, it updates the device-status table and starts the next request.
6. After the ISR completes, the OS restores the saved CPU state and resumes the interrupted program from the instruction where it stopped.

d. Why must the registers and program counter be saved?

The CPU registers and the program counter together form the state of the running program. The program counter holds the address of the next instruction, and the registers hold intermediate data. The ISR uses the same CPU and registers, so without saving them the interrupted program's values would be overwritten and its place would be lost. Saving the state lets the OS restore it after the interrupt and continue the program exactly where it left off, as if it had never been interrupted. This guarantees correct and consistent computing.

Question 3 – Multiprogramming and Process Management [3 Marks]**a. Should the CPU stay idle while P2 waits for disk I/O?**

No. P2 is waiting for I/O and cannot use the CPU, but P1 and P3 are ready, so leaving the CPU idle would waste it. The operating system should switch the CPU to another job. Since P1 is already performing calculations, P1 can continue, or the scheduler can select the ready process P3 through CPU scheduling. The CPU is kept busy while P2's disk operation proceeds, and P2 becomes ready again when the disk controller interrupts on completion.

b. How multiprogramming improves CPU utilization.

A single process cannot keep the CPU and I/O devices busy all the time, since it spends long periods waiting for I/O. Multiprogramming keeps several jobs in memory and organizes them so that the CPU always has one to execute. Job scheduling selects the job to run, and when it has to wait (for example for I/O), the OS switches to another job. Here, P2's wait for the disk overlaps with useful execution of P1 or P3. The CPU therefore does not sit idle while a process

waits, and CPU utilization and throughput increase.

c. P1 enters an infinite loop: the timer mechanism.

An infinite loop would let P1 hog the CPU and prevent P2 and P3 from running. To prevent this, the OS uses a timer. Before scheduling a process, the OS sets the timer to interrupt after a specific period. The counter is decremented, and when it reaches zero the hardware generates a timer interrupt. This transfers control back to the operating system in kernel mode, even though P1 never gives up the CPU voluntarily. The OS can then either terminate P1 because it exceeded its allotted time, or save its state and schedule another process such as P3 or P2. In this way the OS always regains control, and no single process can monopolize the CPU.